

Small Language Model Challenge

Causal Transformer with a within-window repetition cache | Final experimental report | 29 September 2026

Complete-test BPB	CPU time / baseline	Peak RAM	Inference assets
1.5541534308982003	50.982 s / 15.468 s = 3.296x	1.757 GiB	15.04 MiB

1. Task and evaluation protocol

The supplied WikiText-2 raw-text benchmark uses a BPE vocabulary of 2,048 fitted only on training text. The fixed evaluator scores 256-target independent windows, covering every next-token target except the first token of each split. Bits per byte (BPB) is the summed negative base-2 log probability divided by the entire raw UTF-8 byte count. The complete test contains 428,405 targets and 1,292,013 bytes. Final scoring uses CPU FP32. We kept the supplied data, tokenizer, common.py, and evaluate.py unchanged.

Model settings and the final checkpoint were selected using validation. The final test score was observed and later reproduced for timing and RAM; subsequent exploratory architectures were compared on validation and were not selected on their test outcomes. No external corpus, pretrained parameters, evaluation-network access, or cross-window state was used.

2. Model and training

The selected predictor has eight pre-normalized Transformer blocks with width 192, six heads (head dimension 32), RMSNorm, rotary position embeddings, SwiGLU feed-forward layers (hidden width 512), and tied token/output embeddings. Residual-branch dropout is 0.10 in training and off at evaluation. It has 3,935,424 learned parameters. Its model interface returns causal unnormalized logits for training and normalized log probabilities for scoring.

Training starts from random initialization with seed 17. The unmodified trainer uses AdamW, initial learning rate 0.001, weight decay 0.1, 100-step linear warmup, cosine decay to 10% of the base rate, batch size 32, context 256, gradient clipping at norm 1, and BF16 CUDA autocast. The selected run has 16,000 updates, 131,072,000 processed targets, and 1,064.62 s measured GPU training time on an RTX 4060 Laptop GPU. The final weights are from the last update; intermediate validation weights were not saved.

Within-window cache

For each prediction position, the cache looks for earlier occurrences of the current two-token suffix. When none exists, it uses one-token matches with vote weight 0.25. Every prior occurrence votes for its already observed successor token. Votes are normalized into an empirical distribution and mixed with the neural softmax by a mass-dependent coefficient: $\alpha = 0.30 (1 - \exp(-\text{mass} / 2))$. With no match, alpha is zero. A match ending at position j is used only when $j < t$; its successor $j+1$ is therefore already in the prefix at position t . Temporary votes are rebuilt per call and are never carried across windows.

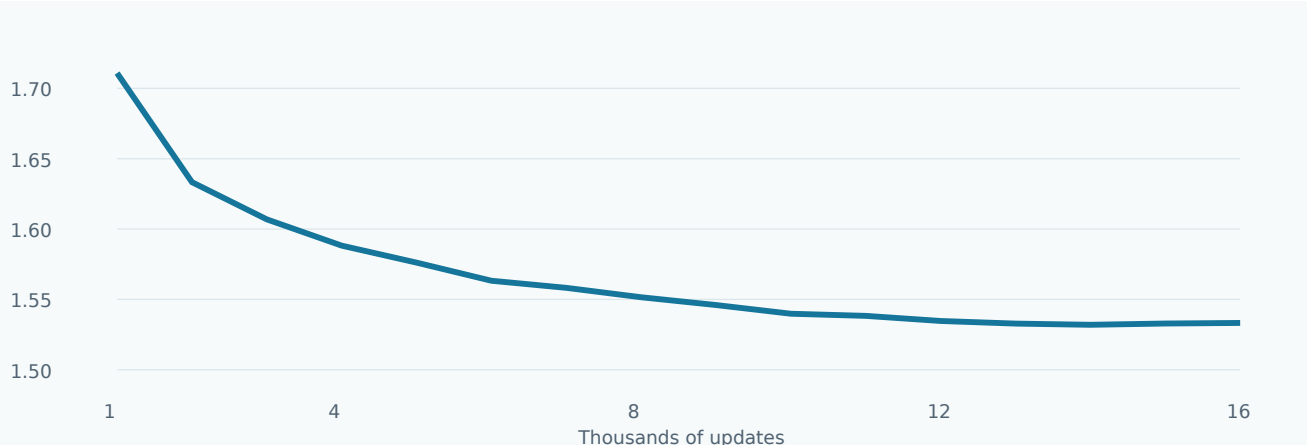
3. Controlled experiments

All quality comparisons below use the same complete validation split and CPU FP32 scorer. The equal-target pair both processes 9,830,400 training targets (1,200 updates x 32 x 256) with seed 17. The full-run ablation loads identical learned weights and changes only the inference cache.

Experiment	Training targets	Validation BPB	CPU seconds
Supplied GPT, 1,200 steps	9,830,400	2.0711885	11.9772
Selected architecture, 1,200 steps; cache on	9,830,400	1.6752045	40.00-43.41
Selected weights, 16,000 steps; cache off	131,072,000	1.5828839	36.8316
Selected weights, 16,000 steps; cache on	131,072,000	1.5332663	validation CPU: see test timing below

At equal processed targets, the student improves validation BPB by 0.3959841 (19.1% relative). Its GPU training took 95.40 s versus 31.36 s for the baseline, about 3.04 times as long. This pair compares the whole model and cache; it does not isolate individual architectural components. At fixed 16,000-step weights, the cache alone improves validation BPB by 0.0496176 (3.14% relative to the no-cache predictor). It increases scoring work but remains within the complete-test CPU budget.

Validation trajectory of the selected run



The curve reaches 1.5319480 at 14,000 updates, then rises slightly. Because the supplied trainer saves only the final checkpoint, the selected reproducible 16,000-step file scores 1.5332663. This plateau argues against simply extending training.

4. Search, failures, and cost

The following measured runs document the local search visible in the experiment logs. All used seed 17 and the supplied training split. Validation governed model decisions; the test split was not used to choose an architecture or cache coefficient. Training seconds omit preparation and intermediate validation.

Run	Updates	Targets	Train s	Best / final validation BPB
Supplied GPT control	1,200	9.83M	31.36	2.07119 / 2.07119
Eight-block equal-target control	1,200	9.83M	95.40	1.67520 / 1.67520
Six-block early model	5,000	40.96M	279.01	1.66134 / 1.66334 without cache
Eight-block candidate	8,000	65.54M	538.61	1.53485 / 1.53485 with cache
Eight-block selected model	16,000	131.07M	1,064.62	1.53195 / 1.53327 with cache
Ten-block candidate	16,000	131.07M	1,190.42	1.52350 / 1.52431 with cache
Eight-block, width 224	16,000	131.07M	1,122.42	1.52682 / 1.53045 with cache

These seven logged training runs total 519,372,800 processed targets and approximately 4,321.84 s of GPU training, excluding preparation and evaluation. Earlier exploratory course runs outside this seven-run log are not quantified here; their logs should be preserved with the repository if available. A direct 50/50 weight average of the eight-block 8,000- and 16,000-step checkpoints gave 1.68747 validation BPB, much worse than either final candidate. The two runs had different cosine schedules despite a shared seed, so parameter averaging was not a reliable substitute for a jointly optimized trajectory.

The ten-block candidate reached 1.52431 validation BPB, but one validation CPU measurement was 69.53 s compared with 11.98 s for the fully trained baseline on that split, above the 5x ratio in that measurement. The width-224 candidate did not improve quality over the ten-block model. The eight-block selected model gives a more robust score-resource balance. Timing varies between repeated local runs, so complete-test timing is reported below using the same split for the two predictors.

5. Complete-test score and resource audit

Resource	Measured value	Constraint
Complete-test BPB	1.5541534308982003	Lower is better
CPU FP32 time	50.9821 s; baseline 15.4677 s; ratio 3.2960x	At most 5x
Peak CPU RAM	1.757 GiB process-tree peak working set	At most 4 GiB
Inference files	15,767,382 bytes = 15.04 MiB (checkpoint + student.py)	At most 64 MiB

A repeat of the same frozen complete-test predictor produced the identical BPB and an evaluator time of 50.11 s. RAM was measured on Windows by sampling the Python launcher and its child process throughout a complete-test CPU FP32 run; peak process-tree working set was 1,886,728,192 bytes. The scorer JSON reports zero for CUDA allocation on CPU, which is not a RAM measurement. The supplied tokenizer belongs to the fixed benchmark package; the listed inference files are the submitted checkpoint and implementation.

6. Critical analysis and reproducibility

The strongest identified gain is the causal within-window cache. It helps repeated phrases and names inside a 256-token window, but cannot recall material from preceding windows. Its suffix-matching work grows quadratically with window length and its dense interpolation uses 2,048 output probabilities at each position. The architecture and longer training also matter, but the equal-target experiment changes several features at once, so it does not attribute their individual effects. Increased width and depth showed diminishing validation returns and could exhaust CPU time. The 1.55415 test BPB is a reproducible score under the measured limits, not evidence that the model reaches a 1.5 BPB target.

Final artifacts are the exact eight-block student.py with SHA-256

f1b62a1f8aaa08c6cc086035994ad42f3f9aa2bbf89e942a3caa78e1314c742e and the 16,000-step checkpoint with SHA-256 **f69540994e0864701a3c854384b1cf1a36faadc5c7d89313a77296265ecd8871**.

The evaluator SHA-256 is 128bcb2dab0be0d427505bddb4671e3ab3a8f78e114be79a689c0f9029af133d; tokenizer SHA-256 is 020d1bc6aa4449c4f352b2e03d0e0fb4f39287f15297705e421b1fa7d817262e. The checkpoint declares implementation="student" and loads with the unmodified scorer.

Reproduction instructions

Use Python 3.12, PyTorch 2.7.1, and the supplied requirements and benchmark package. Copy the exact student.py into code/ and the checkpoint into runs/student-deep-16000/checkpoint.pt. From code/, execute:

```
python evaluate.py --checkpoint runs/student-deep-16000/checkpoint.pt --device cpu --precision fp32 --split test --output reproduced_test.json
```

The complete test JSON must report 428,405 targets, 1,292,013 UTF-8 bytes, and BPB 1.5541534308982003 (small platform-level FP32 differences may occur). The training command used was:

```
python train.py --implementation student --steps 16000 --batch-size 32 --seed 17 --eval-every 1000 --device cuda --run-dir runs/student-deep-16000
```

The default trainer has no optimizer-state resume. Do not substitute the 8,000-step or averaged checkpoint. A reviewer can use the provided scorer directly without retraining. Keep the data and tokenizer manifest unchanged.

Assistance, provenance, and references

Substantive AI assistance was used to draft and revise the Transformer/cache implementation, experiment commands, measurement helper, and this report. The student ran the experiments, supplied the logs, selected the frozen predictor, and is responsible for the final code, claims, and submission. The initial GPT, fixed evaluation protocol, and dataset packaging are course-provided. WikiText-2 was introduced by S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel Mixture Models," arXiv:1609.07843. The dataset text is attributed to Wikipedia contributors under the notices in the supplied README.